

Diretrizes do Trabalho: Validações de Regras de Negócio e Segurança na API

1. Contexto do Trabalho

Você está recebendo o esqueleto de uma API de gerenciamento de vendas e faturas pronta e rodando. As tabelas do banco de dados (PostgreSQL), os services (onde ficam as queries e o banco) e as rotas já foram criados. Porém, os arquivos dentro da pasta **controllers** estão totalmente sem validações de dados e sem nenhuma camada de proteção.

O seu papel neste trabalho é analisar as regras de negócio descritas abaixo e codificar todas as validações necessárias dentro dos controllers, garantindo a consistência das informações e protegendo a API contra requisições erradas ou maliciosas.

2. Regras de Negócio a Implementar

As seguintes regras devem ser validadas nos controllers antes que os dados sejam enviados para a camada de service para serem salvos:

2.1. Validação de Dados Obrigatórios

- **Requisito:** O sistema não pode aceitar a criação de registros em Vendas ou Faturas se faltarem campos obrigatórios no payload enviado. Se algum dado essencial não for enviado, a API deve responder informando quais campos estão faltando.

2.2. Consistência de Valores Financeiros

- **Requisito:** O valor total de uma venda precisa ser um número real e maior do que zero. Impeça que valores zerados, negativos ou formatos de texto inválidos passem para o banco de dados.

2.3. Paridade entre Venda e Fatura

- **Requisito:** Ao criar uma nova fatura, o valor dela precisa ser exatamente o mesmo valor total da venda associada. Além disso, por questões de compliance, o sistema não pode permitir o cadastro de faturas com datas de vencimento no passado.

2.4. Garantia de Existência e Imutabilidade

- **Requisito:** Se o cliente tentar atualizar um registro passando um ID que não existe na URL, a API deve avisar claramente que o registro não foi localizado. Outro ponto crítico: se uma fatura já estiver com o status igual a 'pago', ela não pode mais ser alterada. Qualquer tentativa de alteração deve ser bloqueada de cara.

2.5. Controle de Datas conforme o Status

- **Requisito:** A alteração do status de uma fatura dita o preenchimento da data de pagamento. Se o status mudar para ' pago ', o campo de data de pagamento deve ser preenchido (use a data atual do servidor se nenhuma data for enviada). Se o status mudar para pendente ou cancelado, force o campo de data de pagamento para ficar nulo (null).

2.6. Autenticação e Controle de Acesso

- **Requisito:** As rotas de vendas e faturas não podem ser acessadas por qualquer um. Todas as requisições para esses endpoints devem exigir a validação do Token de identificação no cabeçalho. Pedidos anônimos ou com tokens inválidos devem ser rejeitados imediatamente.

2.7. Autorização por Perfil Administrativo

- **Requisito:** A exclusão de uma venda é considerada uma ação muito crítica. Por isso, apenas usuários administradores (admin) podem deletar uma venda do sistema. Se um usuário comum (padrao) tentar fazer isso, a requisição deve ser barrada.

2.8. Isolamento de Dados por Usuário

- **Requisito:** Os usuários comuns (padrao) só podem visualizar as faturas vinculadas à sua própria conta. Eles não podem ter acesso às faturas de outras pessoas. O único perfil que tem permissão para listar e auditar todas as faturas do sistema de uma vez é o administrador (admin).

3. Fluxo de Contas e Autenticação

Para interagir com as rotas protegidas por segurança e testar as permissões de acesso, você precisará de duas contas com perfis diferentes. Siga os passos abaixo no Postman ou Insomnia:

3.1. Passo 1: Criando as Contas (Registro)

Faça duas requisições do tipo POST separadas para registrar duas contas distintas. Por padrão, a rota de registro cria os usuários com o nível de acesso comum (padrao).

Rota: POST /auth/register .

Payload do Usuário Padrão:

```
{
  "nome": "Fulano Cliente",
  "email": "cliente@sistema.com",
  "senha": "Senha123"
}
```

Payload do Futuro Administrador:

```
{
  "nome": "Ciclano Admin",
  "email": "admin@sistema.com",
  "senha": "Senha123"
}
```

```
"senha": "Senha123"
}
```

3.2. Passo 2: Alterando para Administrador Direto no Banco

Como a rota de cadastro cria as contas apenas no perfil básico, abra sua ferramenta de gerência de banco (pgAdmin / DBeaver) e mude o privilégio da segunda conta manualmente executando o comando SQL abaixo:

```
UPDATE usuarios
  SET tipo_acesso = 'admin'
 WHERE email = 'admin@sistema.com';
```

3.3. Passo 3: Efetuando o Login para obter o Token

Com as duas contas prontas, faça login enviando as credenciais da conta que você quer testar no momento. A API retornará o Token JWT correspondente ao perfil.

Rota: `POST /auth/login`.

Exemplo de corpo enviado:

```
{
  "email": "admin@sistema.com",
  "senha": "Senha123"
}
```

JSON recebido no sucesso:

```
{
  "status": "ok",
  "token": "REPLACE_BY_LONG_JWT_TOKEN_STRING..."
}
```

3.4. Passo 4: Enviando o Token nas Requests

Copie o token retornado. Para qualquer rota protegida (vendas ou faturas), adicione o código nos cabeçalhos HTTP da requisição:

- No Postman ou Insomnia, acesse a aba **Auth**, mude o tipo para **Bearer Token** e cole o token lá.
- Isso criará de forma automática o header `Authorization: Bearer SEU_TOKEN_AQUI`.

4. Como Configurar e Rodar o Projeto

1. Instalar as dependências:

```
npm install
```

2. Configurar as Variáveis de Ambiente:

Crie um arquivo chamado `.env` na raiz do projeto e adicione as credenciais do seu banco e a chave simétrica seguindo o modelo exato abaixo:

```
PORT=3000
POSTGRES_USER=seu_usuario_postgres
POSTGRES_HOST=localhost
POSTGRES_DB=nome_do_seu_banco
POSTGRES_PASS=sua_senha_postgres
POSTGRES_PORT=5432
AUTH_KEY=sua_chave_secreta_jwt_aqui
```

3. Provisionamento do Banco de Dados:

Abra o pgAdmin ou DBeaver e execute integralmente o script SQL localizado em `src/configs/banco.sql` do projeto para criar a estrutura inicial (tabelas, enums e relacionamentos).

4. Iniciar o Servidor em Ambiente de Desenvolvimento:

```
npm run dev
```

5. Padrão de Resposta Esperado

As respostas JSON que você codificar nas novas validações dos controllers devem seguir o padrão estrutural já existente na aplicação:

Em caso de Sucesso (Ex: 200, 201):

```
{
  "status": "ok",
  "message": "Operação realizada com sucesso",
  "data": { ... }
}
```

Em caso de Erro (Ex: 400, 401, 403, 404, 500):

```
{
  "status": "error",
  "message": "Mensagem amigável explicando o erro ocorrido"
}
```